الجامعة الدولية للرباط
ⵜⴰⵙⴷⴰⵡⵜ ⵜⴰⵖⵔⴰⵡⵜⵜ ⵏ ⵕⵕⴱⴰⵟ
Université Internationale de Rabat
L'UNIVERSITÉ INNOVANTE

Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

Cloud Foundations

Chapter 7-Part 2: Serverless Functions

PR. OUASSIM KARRAKCHOU

UNIVERSITE INTERNATIONALE DE RABAT

Academic year: 2026-2027

Serverless Computing

The example of Azure Functions

- Azure Functions provide a serverless compute experience. A function is invoked by a trigger (such as access to an HTTP endpoint or a timer) and executes a block of code or business logic. Functions also support specialized bindings that connect to resources like storage and queues.

- The following languages are all supported:

Language	Supported runtimes for 4.x
C#	.NET 6.0, 7.0, .NET Framework 4.8
JavaScript	Node 14, 16, 18
F#	.NET 6.0, 7.0
Java	Java 8, 11, 17
Python	Python 3.7, 3.8, 3.9, 3.10
TypeScript	Node 14, 16, 18 (via JavaScript)
PowerShell	PowerShell Core 7.2



Serverless Computing

The example of Azure Functions

- Functions are backed by an app service plan.
- The plan defines the resources used by the functions app. You can assign plans to a region, determine the size and number of virtual machines that will be used, and pick a pricing tier.
- For a true serverless approach, function apps may use the consumption plan. The consumption plan will scale the back end automatically based on load.
- Another hosting option for function apps is the Premium plan. This plan provides an "always on" instance to avoid cold start, supports advanced features like VNet connectivity, and runs on premium hardware.



Serverless Computing

Azure Functions Triggers and Binding

- Functions are invoked by a trigger and can have exactly one.
- You may also define multiple bindings in addition to the trigger. Bindings provide a declarative way to connect data to your code. They can be passed in (input) or receive data after the function is executed (output).
- In addition to invoking the function, certain triggers also serve as bindings (e.g., body of an HTTP request that triggers a function).
- Triggers and bindings make functions easy to work with. Bindings remove the overhead of manually creating database or file system connections.
- All of the information needed for the bindings is contained in a special functions.json file for scripts or declared with attributes in code.



Serverless Computing

Azure Functions Triggers and Binding

- Some common triggers include:
  - Blob Storage: invoke your function when a file or folder is uploaded or changed in storage.
  - HTTP: invoke your function like a REST API.
  - Queue: invoke your function when items exist in a queue.
  - Timer: invoke your function on a regular cadence.
- Examples of bindings include:
  - CosmosDB: easily connect to the database to load or save files.
  - Table Storage: work with key/value storage from your function app.
  - Queue Storage: easily retrieve items from a queue, or place new items on the queue.



Serverless Computing

Azure Functions Triggers and Binding

- The following example functions.json file defines a trigger and a binding:

```
{
  "bindings": [
    {
      "name": "myBlob",
      "type": "blobTrigger",
      "direction": "in",
      "path": "images/{name}",
      "connection": "AzureWebJobsStorage"
    },
    {
      "name": "$return",
      "type": "queue",
      "direction": "out",
      "queueName": "images",
      "connection": "AzureWebJobsStorage"
    }
  ],
  "disabled": false
}
```

- In this example, the function is triggered by a change to blob storage in the images container. The information for the file is passed in, so the trigger also acts as a binding.

- Another binding exists to put information onto a queue named images.



Serverless Computing

Disadvantages of serverless computing

- It may seem that compared to traditional server designs, serverless computing offers three unbeatable advantages:
  - The ability to scale arbitrarily
  - No need to manage servers
  - Lower overall cost
- Despite its advantages, serverless computing does have potential disadvantages.

Based on what you now know about serverless computing, what are its disadvantages?



Serverless Computing

Disadvantages of serverless computing

- Serverless systems introduce latency:
  - A traditional server always starts before any client initiates contact. Thus, the server can respond to requests immediately.
  - In a serverless situation, however, management software launches instances of the server as needed.
  - The delay in starting a server may seem insignificant but can be important for a server that needs to respond quickly (e.g., to handle an event or a transaction).



Serverless Computing

Disadvantages of serverless computing

- Serverless computing can also generate unexpected costs:
  - To see why, consider serverless computing used with a disaggregated microservice. The serverless approach makes it easy to divide processing into small pieces that each run independently.
  - In fact, the concept of Function as a Service can lure software engineers into running each function as a separate piece. Because a provider charges for each instance, however, dividing a program into tiny pieces means the user will incur a small charge for each piece.
  - For example, suppose a program contains four functions A, B, C, and D and for typical input A calls B, B calls C, and C calls D:

$$A \rightarrow B \rightarrow C \rightarrow D$$



Serverless Computing

Disadvantages of serverless computing

- Serverless computing can also generate unexpected costs:

$$A \rightarrow B \rightarrow C \rightarrow D$$

  - If each function runs as a separate entity, a customer will be charged to create four instances whenever A is invoked.
  - Placing all four in the same piece means a customer will only pay for one instance.
  - Although the charge for an instance is trivial, a microservice architecture allows a given microservice to be invoked thousands of times per second, in which case a factor of four in cost can accumulate quickly.



Serverless Computing

Disadvantages of serverless computing

- Error management in serverless computing:

$$A \rightarrow B \rightarrow C \rightarrow D$$

  - We know that serverless functions work using an event-based approach.
  - One category of events are triggered when a serverless function fails.
  - Error reporting and monitoring functions can be created to manage these failure events. For example, to notify an administrator and restart the failed function if needed.
  - What would happen if function D above fails?



Serverless Computing

Disadvantages of serverless computing

- Error management in serverless computing:

$$A \rightarrow B \rightarrow C \rightarrow D$$

  - What would happen if function D above fails?
  - C will send an alert event saying that D has failed and will respond to B that it cannot fulfill the request. B will then send an alert event saying that C failed and will report failure to A.
  - The point is that a failure at a low level can result in a cascade of alerts. Because microservices can be invoked thousands of times per second, the system may receive an unexpected flood of events.
  - If the alert system employs the serverless approach, the system will scale out by creating an instance for each alert. Thus, the charges incurred to scale the system can become surprisingly high.



Serverless Computing

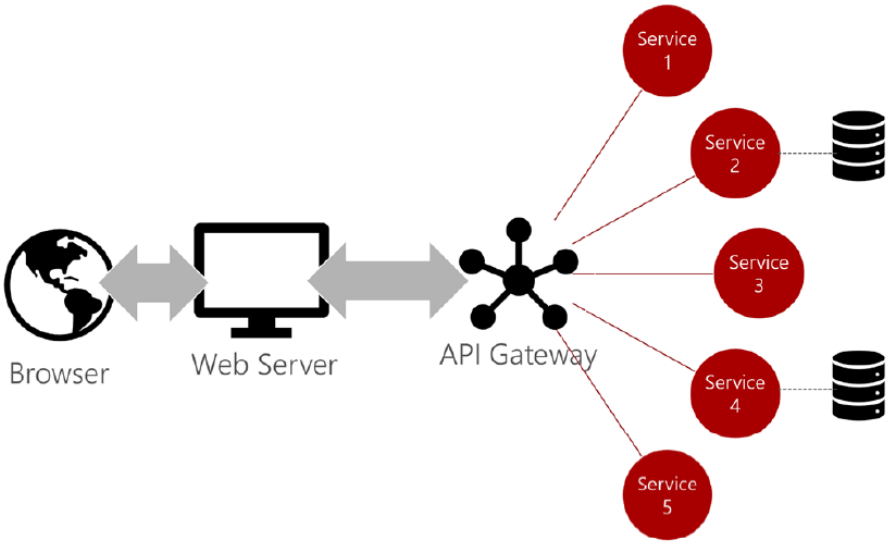
Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Full serverless backend:
  - The full serverless back end is ideal for several types of scenarios, especially when building new or “green field” applications.
  - An application with a large surface area of APIs may benefit from implementing each API as a serverless function.
  - Apps that are based on microservices architecture are another example that could be implemented as a full serverless back end. The microservices communicate over various protocols with each other.

Serverless Computing

Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Full serverless backend:



  - Each service or API becomes a serverless function



Serverless Computing

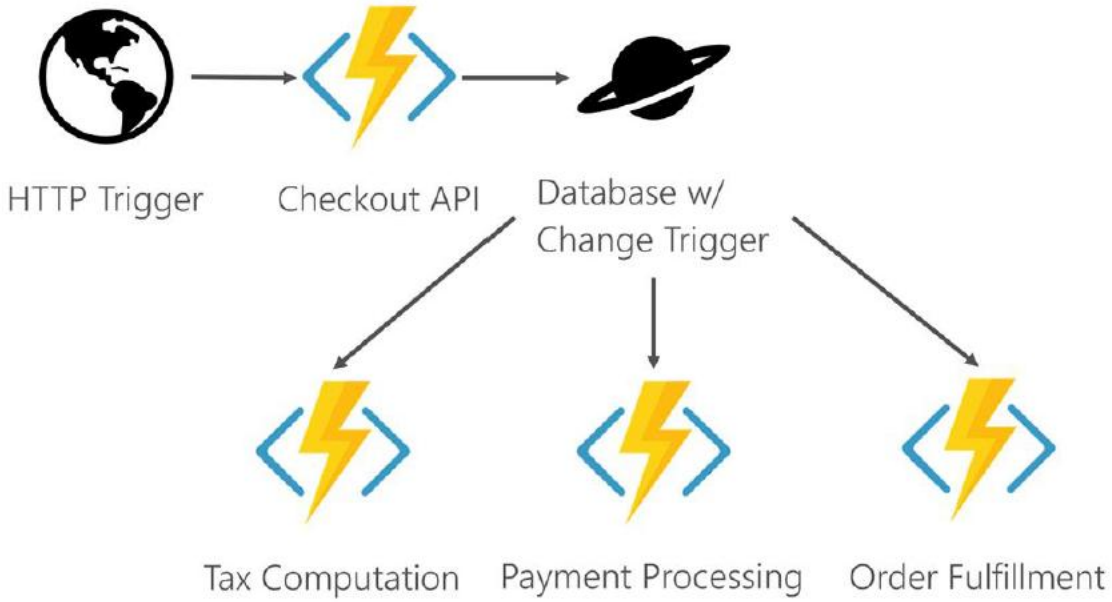
Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
    - Full serverless backend: Examples of application and use cases that can benefit from a full serverless backend are:
        - API-based SaaS products (example: financial payments processor).
        - Message-driven applications (example: device monitoring solution).
        - Apps focused on integration between services (example: airline booking application).
        - Processes that run periodically (example: timer-based database clean-up).
        - Apps focused on data transformation (example: import triggered by file upload).
        - Extract Transform and Load (ETL) processes.

Serverless Computing

Examples of application architectures using Serverless

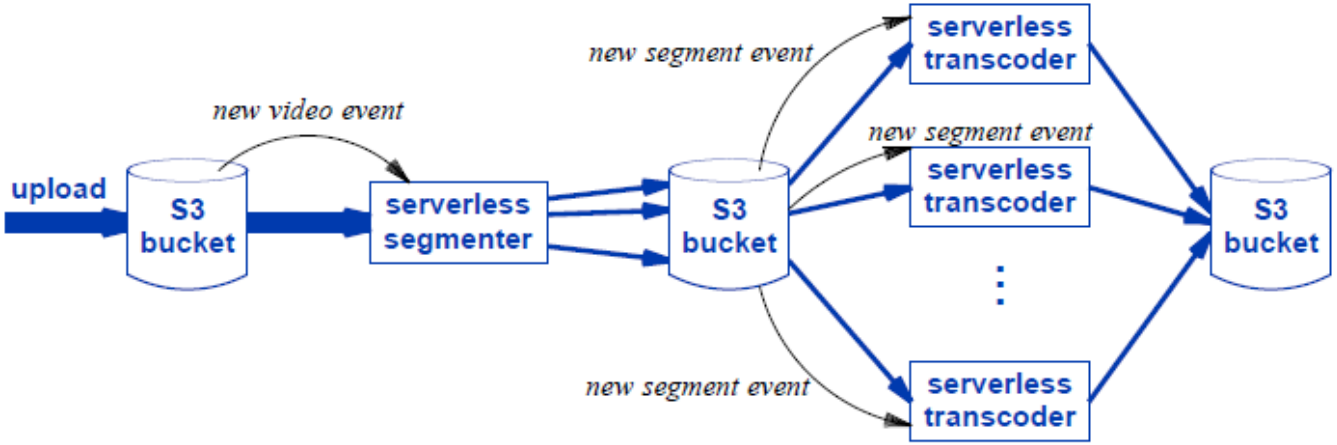
- We will explore in the next few slides some common architecture examples that use serverless :
  - Full serverless backend: Examples of an e-commerce app:



Serverless Computing

Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Full serverless backend: Examples of the Netflix transcoding pipeline:





Serverless Computing

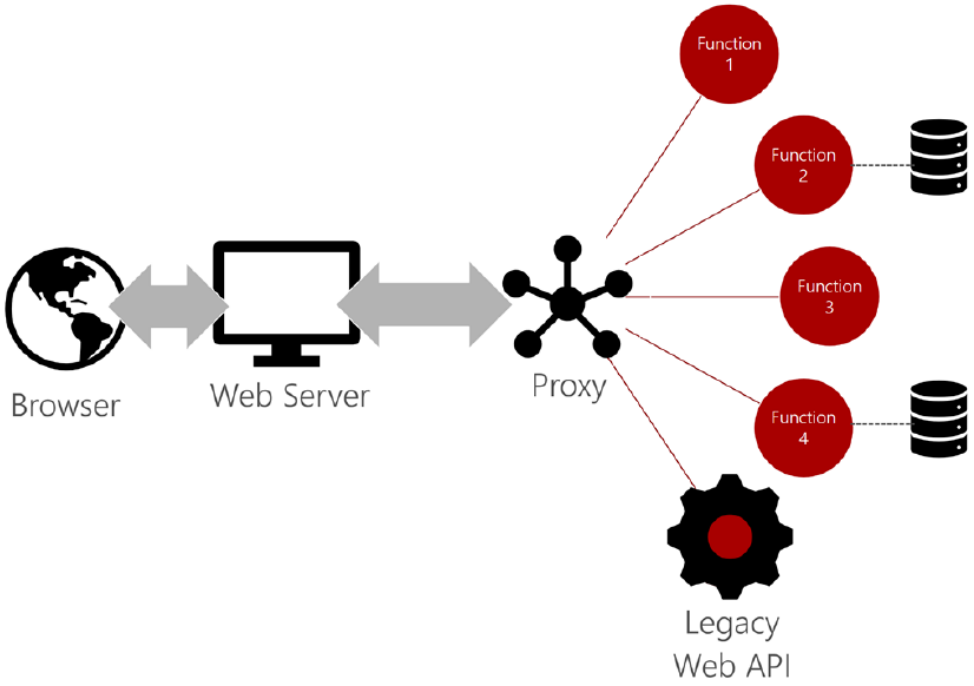
Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Monoliths and “starving the beast”:
  - A common challenge is migrating an existing monolithic application to the cloud.
  - The least risky approach is to “lift and shift” entirely onto virtual machines.
  - Many shops prefer to use the migration as an opportunity to modernize their code base.
  - A practical approach to migration is called “starving the beast.”

Serverless Computing

Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Monoliths and “starving the beast”:





Serverless Computing

Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Monoliths and “starving the beast”:
  - In this scenario, the monolith is migrated “as is” to start with. Then, selected services are modernized. In some cases, the signature of the service is identical to the original: it simply is hosted as a function.
  - Clients are updated to use the new service rather than the monolith endpoint. In the interim, steps such as database replication enable microservices to host their own storage even when transactions are still handled by the monolith.
  - Eventually, all clients are migrated onto the new services. The monolith is “starved” (its services no longer called) until all functionality has been replaced. The combination of serverless and proxies can facilitate much of this migration.



Serverless Computing

Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Web apps:
  - Web apps are great candidates for serverless applications. There are two common approaches to web apps today: server-driven, and client-driven (such as Single Page Application or SPA).
  - Server-driven web apps typically use a middleware layer to issue API calls to render the web UI.
  - SPA applications make REST API calls directly from the browser.
  - In both scenarios, serverless can accommodate the middleware or REST API request by providing the necessary business logic.



Serverless Computing

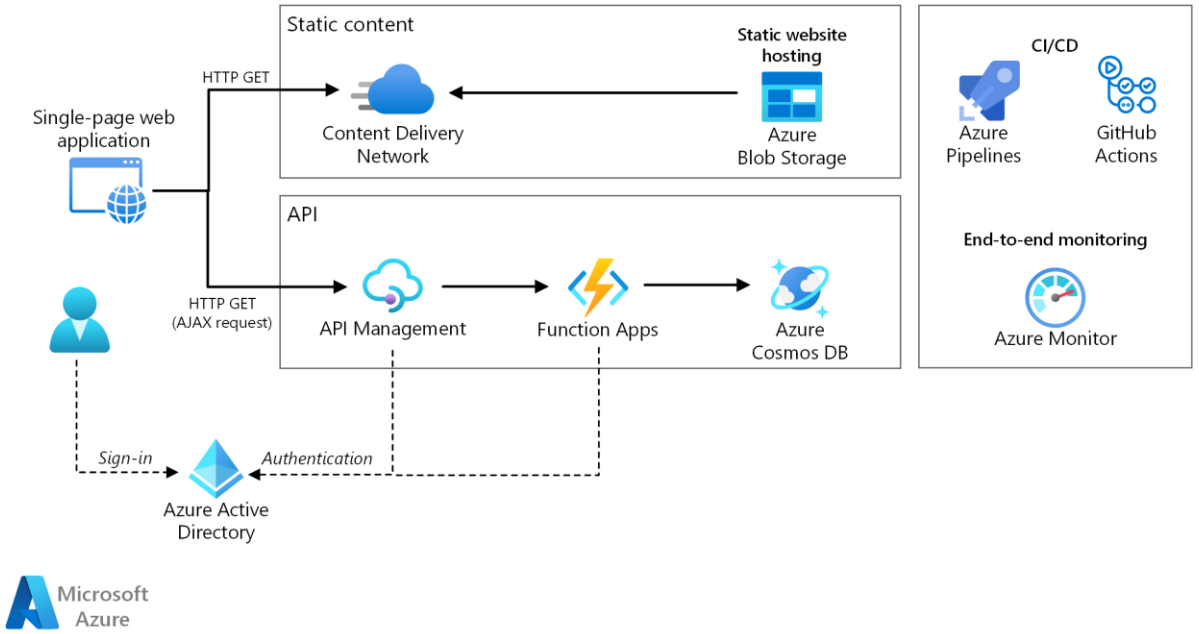
Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Web apps:
  - A common architecture is to stand up a lightweight static web server to serve the Single Page Application (SPA) HTML, CSS, JavaScript, and other browser assets.
  - The web app then connects to a microservices back end.

Serverless Computing

Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - Web apps:





Serverless Computing

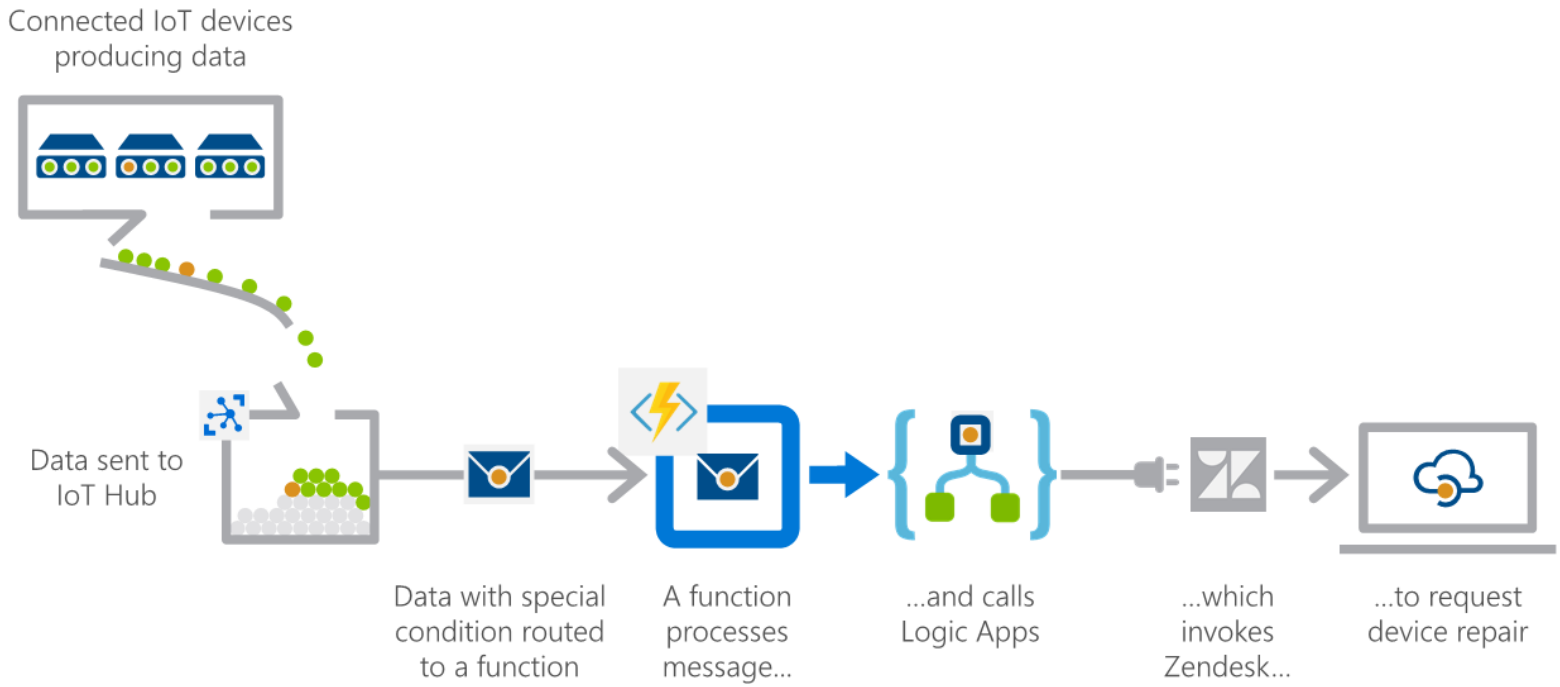
Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - IoT apps:
  - IoT refers to physical objects that are networked together. They're sometimes referred to as "connected devices" or "smart devices."
  - The sheer volume of devices and information often dictates an event-driven architecture to route and process messages. Serverless is an ideal solution for several reasons:
    - Enables scale as the volume of devices and data increases.
    - Accommodates adding new endpoints to support new devices and sensors.
    - Resiliency and less downtime.

Serverless Computing

Examples of application architectures using Serverless

- We will explore in the next few slides some common architecture examples that use serverless :
  - IoT apps:





Containers

Summary

- *Azure functions are triggered by an event and can have a binding to a resource.*
- *Azure functions are managed by an app service plan that handles the pricing and the automatic scaling strategy.*
- *Despite its many advantages, the serverless paradigm has disadvantages of introducing a small delay when starting a server and the potential for unexpected cost to handle large numbers of invocations.*
- *Serverless can be used for many applications, including full serverless backend applications, web apps, and IoT.*
- *Traditional monolithic applications can also be migrated progressively to serverless using a "starve the beast approach".*